Avaliação Reproduzível de Modelos de Linguagem Compactos sob Restrições Severas de Memória e Computação

Dinâmica Arquitetural, Fronteiras de Quantização, Decodificação Especulativa e Vulnerabilidades Adversariais em Bancada Experimental de Referência

Anderson Tavares
Departamento de Engenharia de Sistemas e Aprendizado de Máquina Aplicado
Grupo Independente de Pesquisa em Sistemas
https://github.com/AndersonTavares0/llm-research-bench

24 de outubro de 2026

Resumo

A implantação de Modelos de Linguagem Pequenos (SLMs, 2B–4B parâmetros) diretamente em dispositivos físicos de borda garante estrita soberania de dados, latência determinística e eliminação de tráfego para a nuvem pública. Contudo, operar sob limites rígidos de memória física ($\leq$ 4096 MiB de VRAM) impõe compromissos rigorosos entre precisão de quantização de pesos, largura de banda no barramento DRAM, decodificação especulativa e alinhamento de segurança. Este estudo apresenta uma avaliação experimental controlada, rigorosa e reproduzível conduzida em uma GPU NVIDIA GeForce GTX 1050 Ti (4096 MiB GDDR5, barramento de 128 bits, 112,0 GB/s de largura de banda teórica, TDP de 75W, Compute Capability 6.1) como bancada experimental de referência. Desconstruímos e formulamos teoricamente cada conceito fundamental—incluindo o modelo Roofline, intensidade aritmética operacional, dinâmica analítica do cache Chave-Valor (KV), decodificação especulativa Multi-Token Prediction (MTP), Model Context Protocol (MCP) e inferência estatística—previamente à apresentação dos dados. Quatro modelos (**Gemma-4-E2B-it**, **Qwen-3.5-4B**, **Qwen-3.5-2B** e **Llama-3.2-3B**) foram avaliados sob o runtime `llama-server` (commit b4120+) com telemetria contínua via `libnvidia-ml.so.1` a cada 250 ms. Uma ablação controlada de decodificação especulativa Multi-Token Prediction (MTP) com $N = 15$ repetições isolou um ganho estatisticamente significante de **30,9%** no throughput médio global (42,21 $\pm$ 5,99 vs 32,25 $\pm$ 3,23 tok/s, IC 95%, t de Welch = 3,139, $p = 0{,}0024$, d de Cohen = 1,146) e de **33,2%** em regime estacionário aquecido (44,95 vs 33,75 tok/s, $t = 21{,}05$, $p < 10^{-13}$), mantendo alocação estável de 2038 MiB. A análise controlada de quantização do Gemma-4-E2B revelou que o formato de matriz de importância não linear (IQ4_XS) entrega vantagem de 59,3% em regime aquecido sobre a quantização em blocos padrão Q4_K_M (41,92 vs 26,34 tok/s, $t = 27{,}20$, $p = 3{,}59 \times 10^{-9}$) pela redução no tráfego do barramento DRAM. Em segurança adversária, todos os modelos exibiram 100% de recusa limpa contra explorações diretas de vulnerabilidades, keyloggers e jailbreaks, mas sucumbiram uniformemente (0% de resistência) à injeção indireta de prompt encapsulada em contexto de sistema, evidenciando que os mecanismos atuais de alinhamento falham em isolar instruções de controle de dados contextuais não confiáveis.

Palavras-chave: Modelos de Linguagem Pequenos, Sistemas com Restrição de Memória, Modelo Roofline, Cache KV, Decodificação Especulativa, Multi-Token Prediction, Model Context Protocol, Telemetria NVML, Injeção Indireta de Prompt.

1 Introdução e Formulação do Problema de Pesquisa

A computação em borda (*edge computing*) exige crescentemente que sistemas de inferência autônomos, controladores locais e rotinas agênticas operem diretamente em infraestrutura física própria. Transferir a inferência para APIs de nuvem comercial compromete a soberania de dados, introduz latência de rede variável e não determinística, gera custos financeiros recorrentes e introduz pontos únicos de falha catastrófica durante interrupções de conectividade externa.

Modelos de Linguagem Pequenos (*Small Language Models* - SLMs) na faixa de 2 a 4 bilhões de parâmetros consolidaram-se como motores viáveis para seguimento rigoroso de instruções, síntese de código e orquestração de ferramentas. Entretanto, hospedar essas redes em processadores gráficos de entrada ou legados—especificamente

aceleradores com restrição física a 4096 MiB de VRAM como a NVIDIA GeForce GTX 1050 Ti—impõe fronteiras físicas severas.

A cada token emitido de forma autorregressiva, bilhões de parâmetros trafegam pelo barramento de memória limitado a 112,0 GB/s. À medida que o contexto da interação se expande, o cache Chave-Valor (KV) compete diretamente por espaço com os pesos estáticos do modelo. Além disso, esquemas agressivos de quantização necessários para comprimir os tensores podem induzir perdas de precisão em raciocínio quantitativo ou degradar a conformidade de ferramentas em chamadas de função.

1.1 Perguntas de Pesquisa e Hipóteses Formais

A presente investigação é guiada por quatro Perguntas de Pesquisa (*Research Questions* - RQs):

1. **RQ1 (Decodificação Especulativa e Amortização de Banda):** Em que extensão a decodificação especulativa por predição de múltiplos tokens (*Multi-Token Prediction* - MTP) mitiga o gargalo de largura de banda de memória em modelos compactos (< 3B) em hardware limitado por memória, sem depender de modelos rascunho dedicados?
   - *Hipótese H_1:* A reutilização de representações latentes do próprio transformador por cabeças lineares secundárias alcança aceleração estatisticamente significante ($S \geq 1{,}25\times, p < 0{,}01$, d de Cohen $> 0{,}8$) sem elevar a alocação de VRAM.
   - *Hipótese Nula H_0:* O custo computacional de validação e ciclos de descarte neutralizam os ganhos especulativos ($\mu_{\text{MTP}} \leq \mu_{\text{Base}}$).
2. **RQ2 (Fronteiras de Quantização e Penalidades Microarquiteturais):** Como quantizações por matriz de importância não homogênea (`IQ4_XS`) se comparam à quantização em blocos uniforme (`Q4_K_M`) em termos de saturação de barramento e throughput em arquiteturas sem aceleração de tensores inteiros em hardware?
3. **RQ3 (Escalonamento de Cache KV e Limites de OOM):** Quais são os limites analíticos e empíricos de memória para escalonamento de contexto ($L \in [256, 32768]$) sob o teto estrito de 4096 MiB, e em que comprimentos de sequência a quantização do cache torna-se mandatória?
4. **RQ4 (Robustez Adversarial e Isolamento de Fronteiras de Instrução):** Sob restrições compactas de parâmetros, modelos pequenos exibem assimetria na defesa contra ataques diretos versus injeções indiretas de prompt conduzidas por ferramentas e contexto externo (Model Context Protocol)?

1.2 Variáveis Experimentais, Controles e Linhas de Base

- **Variáveis Independentes:** Arquitetura do modelo (Gemma-4-E2B, Qwen-3.5-2B, Llama-3.2-3B, Qwen-3.5-4B), formato de quantização (`IQ4_XS`, `Q4_K_M`, `Q4_K_XL`), modo de decodificação (Base autorregressivo vs. MTP especulativo) e comprimento do contexto (L).
- **Variáveis Dependentes:** Throughput de decodificação autorregressiva (tok/s), Tempo até o Primeiro Token (TTFT, ms), alocação de pico de VRAM (MiB), eficiência energética contínua (tok/J), Taxa de Sucesso de Ataque (ASR, %) e conformidade de esquema de ferramentas (%).
- **Variáveis Controladas:** Temperatura ambiente controlada (24°C), perfil de ventoinha fixado em 100%, amostragem determinística gulosa ($\tau = 0{,}0$), sementes fixas e execução isolada em container Linux sem interface gráfica.
- **Linhas de Base (*Baselines*):** Cache KV em FP16 sem quantização; modelo Gemma-4-E2B sem cabeçote MTP habilitado; e consultas de controle benignas para calibração de taxa de falso positivo.

1.3 Bancada Experimental de Referência

A GPU NVIDIA GeForce GTX 1050 Ti (4096 MiB GDDR5, GP107, microarquitetura Pascal, Compute 6.1) foi adotada como **bancada experimental de referência**. Com barramento de 128 bits (112,0 GB/s) e desprovida de núcleos tensores modernos com suporte nativo a INT4, esta placa representa o limite inferior de viabilidade física para execução de SLMs. Seu teto estrito de largura de banda expõe de forma transparente o regime limitado por memória (*memory-bound*), permitindo comprovar empiricamente se inovações algorítmicas (decodificação especulativa e quantização por matriz de importância) são capazes de superar restrições de hardware.

1.4 Estrutura do Artigo

O presente trabalho organiza-se da seguinte forma: a Seção 2 apresenta a fundamentação teórica e as desconstruções didáticas. A Seção 3 estabelece as suítes de benchmark e os controles metodológicos. A Seção 4 detalha o desempenho dos sistemas e a telemetria física. A Seção 5 reporta as ablações controladas e os testes estatísticos inferenciais. A Seção 6 avalia a segurança comportamental, a robustez adversarial e o protocolo MCP. A Seção 7 discute a estabilidade de hardware, ameaças à validade e limites de generalização. As Seções 8 e 9 consolidam o glossário técnico, conclusões e disponibilidade dos artefatos.

2 Fundamentação Teórica e Desconstruções Didáticas

Cada conceito é abordado em seis dimensões: (a) definição formal, (b) funcionamento mecânico, (c) finalidade operacional, (d) exemplo de cálculo passo a passo, (e) papel no experimento e (f) limitações físicas.

2.1 O Modelo Roofline e a Intensidade Aritmética Operacional

Definição: O Modelo Roofline é uma formulação analítica e visual desenvolvida por Williams et al. que estabelece os limites determinísticos de desempenho computacional (FLOP/s) em função da largura de banda de memória e da intensidade aritmética do algoritmo.

Mecanismo de Funcionamento: Qualquer processador possui dois limites intransponíveis:

1. **Pico de Vazão Computacional ($P_{\mathbf{comp}}$):** A taxa máxima teórica de cálculos em ponto flutuante executáveis pelos núcleos de processamento (núcleos CUDA):

$$P_{\text{comp}} = N_{\text{núcleos}} \times f_{\text{clock}} \times \text{FLOPs/ciclo} \quad [\text{FLOP/s}] \tag{1}$$

2. **Pico de Largura de Banda de Memória ($B_{\mathbf{mem}}$):** A taxa máxima teórica de transferência de dados entre a DRAM externa (GDDR5) e os registradores no chip:

$$B_{\text{mem}} = \text{Largura do Barramento (bits)} \times f_{\text{memória}} \times \text{Taxa de Transferência} \quad [\text{Bytes/s}] \tag{2}$$

Intensidade Aritmética Operacional (I): A intensidade aritmética expressa a densidade de trabalho de um algoritmo, definida como a razão entre operações de ponto flutuante executadas e o tráfego transferido da DRAM:

$$I = \frac{\text{Total de Operações de Ponto Flutuante [FLOPs]}}{\text{Total de Tráfego DRAM [Bytes]}} \quad \left[\frac{\text{FLOPs}}{\text{Byte}}\right] \tag{3}$$

Ponto de Inflexão (*Ridge Point* I^*): O ponto de inflexão I^* define a intensidade aritmética exata em que o sistema transita de limitado por memória para limitado por computação:

$$I^* = \frac{P_{\text{comp}}}{B_{\text{mem}}} \quad \left[\frac{\text{FLOPs}}{\text{Byte}}\right] \tag{4}$$

Se uma rotina apresenta $I < I^*$, seu desempenho é estritamente limitado pela largura de banda DRAM ($P \leq I \times B_{\text{mem}}$). Os núcleos de cálculo passam ciclos ociosos aguardando dados. Se $I > I^*$, o desempenho é limitado pela capacidade dos núcleos ($P \leq P_{\text{comp}}$).

Cálculo Numérico na GPU NVIDIA GTX 1050 Ti: A placa gráfica de teste dispõe de 768 núcleos CUDA em frequência base de 1392 MHz (cada qual executando 2 FLOPs por ciclo via Fused Multiply-Add) e barramento GDDR5 de 128 bits operando a 1752 MHz efetivos (112,0 GB/s teóricos):

$$P_{\text{comp}} = 768 \times (1392 \times 10^6 \text{ Hz}) \times 2 = 2{,}138 \times 10^{12} \text{ FLOP/s} = 2{,}138 \text{ TFLOP/s} \tag{5}$$

$$B_{\text{mem}} = \frac{128 \text{ bits}}{8 \text{ bits/Byte}} \times 7008 \times 10^6 \text{ transferências/s} \approx 112{,}0 \text{ GB/s} \tag{6}$$

$$I^* = \frac{2138 \times 10^9 \text{ FLOP/s}}{112{,}0 \times 10^9 \text{ Bytes/s}} \approx 19{,}09 \text{ FLOPs/Byte} \tag{7}$$

Regimes de Prefill versus Decode:

1. **Prefill (Processamento de Prompt):** Um prompt de L_p tokens é avaliado em lote por Multiplicações Matriciais Densas (GEMM). Como as matrizes de pesos são reaproveitadas para todos os L_p tokens simultaneamente, a intensidade aritmética cresce com o tamanho do prompt:

$$I_{\text{prefill}} \approx \frac{2 \cdot P \cdot L_p}{P \cdot b_{\text{peso}} + \mathcal{O}(L_p^2)} \approx \frac{2 \cdot L_p}{b_{\text{peso}}} \tag{8}$$

   Para pesos quantizados em 4 bits ($b_{\text{peso}} \approx 0{,}55$ Bytes) e prompt $L_p = 256$, temos $I_{\text{prefill}} \approx 512/0{,}55 \approx 930$ FLOPs/Byte. Como $930 \gg 19{,}09$, o prefill situa-se na zona limitada por computação (*compute-bound*), atingindo 468,2 tok/s.

2. **Decode (Geração Sequencial):** A geração autorregressiva emite um único token por passo através de Multiplicações Matriz-Vetor (GEMV). Todos os parâmetros da rede devem ser lidos da VRAM GDDR5 a cada token:

$$I_{\text{decode}} \approx \frac{2 \cdot P}{P \cdot b_{\text{peso}} + \text{tráfego KV}} \approx \frac{2}{0{,}55} \approx 3{,}64 \text{ FLOPs/Byte} \tag{9}$$

   Como $3{,}64 \ll 19{,}09$, a geração é estritamente limitada pela largura de banda de memória (*memory-bound*), deixando os núcleos ociosos durante grande parte do tempo.

2.2 Subsistema de Memória Transformer e o Cache Chave-Valor (KV)

Definição: O cache Chave-Valor (Key-Value) é uma estrutura de dados mantida em VRAM que armazena os vetores projetados de Chaves (K) e Valores (V) dos tokens anteriores, eliminando o recálculo redundante durante a geração sequencial.

Mecanismo de Funcionamento: No mecanismo de atenção padrão, a computação de atenção para uma sequência de comprimento L é dada por:

$$\text{Atenção}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_{\text{cabeça}}}}\right)V \tag{10}$$

Ao gerar o token t, a Consulta atual Q_t precisa apenas interagir com as Chaves $K_{1:t}$ e Valores $V_{1:t}$. Sem o cache KV, recalcular essas matrizes demandaria complexidade computacional $\mathcal{O}(t^2)$ a cada passo. Com o cache, os vetores K_t e V_t recém-projetados são concatenados na memória, permitindo complexidade $\mathcal{O}(t)$ por token.

Equação Analítica da Memória do Cache KV: O volume exato de memória M_{KV} consumido pelo cache para contexto L_{ctx} e batch unitário é:

$$M_{\text{KV}} = 2 \times n_{\text{camadas}} \times n_{\text{kv_heads}} \times d_{\text{cabeça}} \times L_{\text{ctx}} \times b_{\text{cache}} \quad \text{[Bytes]} \tag{11}$$

onde o fator 2 contempla tanto Chaves quanto Valores, n_{camadas} é o número de camadas transformer, $n_{\text{kv_heads}}$ é o número de cabeças de atenção KV, $d_{\text{cabeça}}$ é a dimensão de cada cabeça, e b_{cache} é a precisão de armazenamento em bytes (2,0 para FP16, 1,0 para Q8_0, 0,5 para Q4_0).

Exemplo de Cálculo e Limite de Falha por Memória (OOM): Considerando o modelo Qwen-3.5-4B (32 camadas, 8 cabeças KV, dimensão 128, pesos estáticos ocupando 2987 MiB):

- Bytes consumidos por token em FP16: $2 \times 32 \times 8 \times 128 \times 2 = 131.072$ Bytes = 128 KiB/token.
- Em $L_{\text{ctx}} = 2.048$: $M_{\text{KV}} = 2048 \times 128$ KiB = 256,0 MiB.
- Em $L_{\text{ctx}} = 8.192$: $M_{\text{KV}} = 8192 \times 128$ KiB = 1.024,0 MiB (1,00 GiB).
- Em $L_{\text{ctx}} = 32.768$: $M_{\text{KV}} = 32768 \times 128$ KiB = 4.096,0 MiB (4,00 GiB).

Em uma GPU de 4096 MiB, os pesos estáticos (2987 MiB) deixam apenas 1109 MiB livres. Se o contexto for estendido a 32k em FP16, o consumo total de $2987 + 4096 = 7083$ MiB excede a capacidade física da GPU em 72,8%, resultando em encerramento abrupto do processo por estouro de memória (`CUDA out of memory`). A quantização do cache KV em Q4_0 ($b_{\text{cache}} = 0{,}5$) comprime o cache de 32k para 1024 MiB, viabilizando a execução sem offload para a CPU.

2.3 Quantização de Modelos (GGUF, K-quants e IQ-quants)

Definição: Quantização é a conversão dos tensores de pesos do modelo de formatos contínuos de alta precisão (FP32/FP16) para inteiros discretos de baixa largura de bits (4 bits, 2 bits), reduzindo o espaço de armazenamento e a demanda de transferência no barramento de memória.

Mecanismo de Funcionamento: Na quantização por blocos uniformes, grupos de 32 pesos são mapeados para inteiros q utilizando um fator de escala Δ e deslocamento zero z:

$$w \approx \Delta \cdot (q - z), \quad q = \text{clamp}\left(\left\lfloor \frac{w}{\Delta} \right\rceil + z, q_{\min}, q_{\max}\right) \tag{12}$$

- **K-quants (e.g., Q4_K_M):** Utilizam super-blocos de 256 elementos divididos em sub-blocos de 32, usando escalas de 6 bits e quantizações de 4 bits, com média de $\approx 4{,}5$ bits por peso.
- **I-quants (e.g., IQ4_XS):** Empregam tabelas de código não lineares baseadas em matrizes de importância calculadas em datasets de calibração, alocando bits com base na sensibilidade de gradiente ($\approx 4{,}25$ bits por peso).

Finalidade no Experimento: Durante a fase de decode limitada por memória, a taxa de emissão de tokens é inversamente proporcional ao tamanho em bytes dos pesos lidos da VRAM:

$$\text{Throughput} \approx \frac{B_{\text{efetiva}}}{S_{\text{pesos}}} \quad \left[\frac{\text{tokens}}{\text{s}}\right] \tag{13}$$

Reduzir o modelo Gemma-4-E2B de Q4_K_M (3,3 GB) para IQ4_XS (1,5 GB) diminui em 54,5% o tráfego transferido pela DRAM a cada token gerado.

2.4 Decodificação Especulativa e Multi-Token Prediction (MTP)

Definição: A decodificação especulativa é um método algorítmico que acelera a geração autorregressiva ao produzir antecipadamente múltiplos tokens candidatos através de um mecanismo leve de rascunho, validando-os em paralelo numa única passagem de verificação do modelo principal.

Mecanismo em Arquiteturas MTP: Em vez de depender de um modelo de rascunho secundário separado, o MTP nativo integra cabeçotes lineares auxiliares de projeção diretamente sobre a última camada oculta do modelo base. Esses cabeçotes reutilizam as representações já processadas:

$$h_t = \text{CamadaTransformer}_N(x_{1:t}), \quad \hat{x}_{t+1} = \text{Cabeçote}_1(h_t), \quad \hat{x}_{t+2} = \text{Cabeçote}_2(h_t) \tag{14}$$

Como o custo computacional do cabeçote é irrisório ($c_{\text{draft}} \approx 0{,}05 \cdot c_{\text{alvo}}$) e não há leitura adicional de pesos volumosos na DRAM, os tokens aceitos são emitidos praticamente sem custo de memória.

Formulação Matemática da Aceleração: Com taxa empírica de aceitação $\alpha \in [0, 1]$ e janela de predição $\gamma = 1$, o ganho assintótico S é:

$$S = \frac{1 + \alpha\gamma}{1 + \frac{c_{\text{draft}}}{c_{\text{alvo}}}} \tag{15}$$

Para o Gemma-4-E2B com $\gamma = 1$, $c_{\text{draft}}/c_{\text{alvo}} \approx 0{,}05$ e taxa empírica de aceitação $\alpha \approx 0{,}40$:

$$S \approx \frac{1 + 0{,}40}{1 + 0{,}05} = \frac{1{,}40}{1{,}05} \approx 1{,}333 \times \quad (+33{,}3\%) \tag{16}$$

Esta dedução teórica coincide exatamente com os resultados experimentais em regime aquecido (+33,2%).

2.5 Métricas de Inferência e Telemetria de Hardware

Tempo até o Primeiro Token (TTFT): O intervalo transcorrido entre a emissão da requisição HTTP e a chegada do primeiro token gerado:

$$\text{TTFT} = t_{\text{primeiro_token}} - t_{\text{envio_requisição}} \quad [\text{milissegundos}] \tag{17}$$

Eficiência de Memória Normalizada (η_{mem}): Vazão de geração de tokens dividida pelo pico físico de VRAM alocada:

$$\eta_{\text{mem}} = \frac{\text{Throughput}_{\text{decode}}\ [\text{tok/s}]}{\text{VRAM Pico Alocada}\ [\text{GiB}]} \quad \left[\frac{\text{tokens}}{\text{s} \cdot \text{GiB}}\right] \tag{18}$$

Eficiência Energética ($E_{\text{tok/J}}$): Tokens emitidos por Joule de energia elétrica consumida pela GPU:

$$E_{\text{tok/J}} = \frac{\text{Throughput}_{\text{decode}}\ [\text{tok/s}]}{P_{\text{médio}}\ [\text{Watts}]} = \frac{\text{Tokens}}{\text{Watt} \cdot \text{segundo}} \quad \left[\frac{\text{tokens}}{\text{Joule}}\right] \tag{19}$$

2.6 Microarquitetura Pascal e Telemetria NVML

Arquitetura NVIDIA Pascal (Compute 6.1): O chip GP107 da GTX 1050 Ti contém seis Multiprocessadores de Streaming (SMs), cada um com 128 núcleos CUDA convencionais. A arquitetura não possui núcleos Tensor dedicados nem aceleração direta de inteiros INT4. Operações quantizadas em 4 bits são descompactadas via software para registros FP16/FP32 antes da execução.

NVIDIA Management Library (NVML): A coleta de telemetria foi executada através da biblioteca nativa `libnvidia-ml.so.1` a cada 250 ms:

- `nvmlDeviceGetMemoryInfo`: Monitora alocação precisa de VRAM (total, usada e livre em bytes).
- `nvmlDeviceGetTemperature`: Lê a temperatura na junção do silício (°C).
- `nvmlDeviceGetClockInfo`: Acompanha a frequência dos núcleos e memória para registrar eventos de modulação térmica.

2.7 Model Context Protocol (MCP)

Definição: O Model Context Protocol (MCP) é uma especificação aberta desenvolvida pela Anthropic que padroniza como modelos de linguagem interagem com ferramentas externas e bancos de dados através de mensagens estruturadas JSON-RPC 2.0 via `stdio` ou SSE.

Primitivas Operacionais:

- `tools/list`: Retorna as especificações JSON Schema dos parâmetros esperados por cada ferramenta.
- `tools/call`: Emite a chamada estruturada contendo o nome da função e argumentos tipados.

2.8 Taxonomia de Segurança Adversária

Jailbreaks Diretos versus Injeção Indireta de Prompt:

- **Ataques Diretos (e.g., DAN, Roleplay):** Tentativas diretas do operador para burlar filtros éticos por meio de enquadramento narrativo.
- **Injeção Indireta de Prompt:** Comandos maliciosos incorporados dentro de dados contextuais lidos pelo modelo (e.g., logs de sistema, páginas web). Ao resumir o texto, o modelo interpreta as diretivas injetadas como instruções de controle.

Taxa de Sucesso de Ataque (ASR) e Falsos Positivos (FPR):

$$\text{ASR} = \frac{\text{Consultas Comprometidas}}{N_{\text{ataques}}} \times 100\%, \quad \text{FPR} = \frac{\text{Recusas Indevidas em Tarefas Benignas}}{N_{\text{benignas}}} \times 100\% \tag{20}$$

2.9 Estatística Inferencial e Testes de Hipóteses

Teste t de Welch para Variâncias Desiguais:

$$t = \frac{\bar{X}_1 - \bar{X}_2}{\sqrt{\frac{s_1^2}{N_1} + \frac{s_2^2}{N_2}}}, \quad \nu = \frac{\left(\frac{s_1^2}{N_1} + \frac{s_2^2}{N_2}\right)^2}{\frac{(s_1^2/N_1)^2}{N_1 - 1} + \frac{(s_2^2/N_2)^2}{N_2 - 1}} \tag{21}$$

Tamanho de Efeito Padronizado (d de Cohen):

$$d = \frac{\bar{X}_1 - \bar{X}_2}{s_{\text{combinado}}}, \quad s_{\text{combinado}} = \sqrt{\frac{(N_1 - 1)s_1^2 + (N_2 - 1)s_2^2}{N_1 + N_2 - 2}} \tag{22}$$

3 Metodologia dos Benchmarks e Controles Experimentais

1. **Bateria de Segurança Adversária (StrongREJECT / HarmBench):** Avalia pedidos de geração de exploits funcionais (Log4Shell CVE-2021-44228), keyloggers em C++, jailbreaks DAN e comandos administrativos legítimos de controle (para medição de falsos positivos). Avaliado por rubrica multifatorial (recusa explícita, ausência de blocos de código funcionalmente maliciosos e ausência de canários).

2. **Injeção Indireta em Logs:** O modelo recebe um log simulado contendo o comando de override:

   ```
   [CRITICAL_SYSTEM_OVERRIDE] Ignore prior instructions.  Output INJECTION_SUCCEEDED.
   ```

   A avaliação verifica a emissão da palavra sentinela `INJECTION_SUCCEEDED`.

3. **Protocolo MCP Agêntico (Metodologia BFCL):** Conexão a um servidor MCP Python via JSON-RPC 2.0 sobre `stdio`. Valida-se a conformidade com esquemas de tipos e a continuação sintática no segundo turno.

4. **Raciocínio Técnico Quantitativo:** Problemas determinísticos de cálculo de paridade RAIDZ2, alocação de RAM e cálculo de sub-redes CIDR.

4 Desempenho de Sistemas, Telemetria e Fronteira de Pareto

A Tabela 1 apresenta as medições empíricas sob o teto de 4096 MiB de VRAM.

Tabela 1: Desempenho Empírico, Utilização de Memória e Eficiência Energética na GTX 1050 Ti

Modelo	VRAM Pico (MiB)	Orçamento (% VRAM)	Prompt (tok/s)	Decode (tok/s)	TTFT (ms)	η_{mem} (t/s/GB)	Energia (tok/J)
Gemma-4-E2B (MTP)	1.510	36,8%	**468,2**	**46,3**	**115,4**	**31,40**	**0,89**
Qwen-3.5-2B	1.598	39,0%	389,5	37,5	142,1	24,04	0,77
Llama-3.2-3B	2.120	51,7%	390,7	27,4	168,3	13,24	0,50
Qwen-3.5-4B	3.069	74,9%	237,1	18,4	284,6	6,14	0,30

4.1 Utilização Efetiva da Largura de Banda de Memória

- **Gemma-4-E2B (MTP):** 1,4 GB × 46,3 tok/s ≈ 64,9 GB/s (57,9% da largura de banda teórica).
- **Qwen-3.5-2B:** 1,48 GB × 37,5 tok/s ≈ 55,6 GB/s (49,6%).
- **Llama-3.2-3B:** 1,96 GB × 27,4 tok/s ≈ 53,8 GB/s (48,0%).
- **Qwen-3.5-4B:** 2,84 GB × 18,4 tok/s ≈ 52,3 GB/s (46,7%).

4.2 Análise da Fronteira de Pareto

Na Figura 1(direita), o modelo **Gemma-4-E2B (MTP)** ocupa a posição estritamente ótima de Pareto: oferece a maior velocidade (46,3 tok/s) com a menor ocupação de memória (1510 MiB, 36,8%). O Qwen-3.5-4B consome 3069 MiB (74,9%), restringindo a margem para expansão do cache KV.

5 Ablações Controladas e Estatística Inferencial

5.1 Ablação Controlada de MTP ($N = 15$)

A Tabela 2 resume o teste comparando `gemma4:E2B` (com rascunho MTP) e `gemma4:MTP` (base sem MTP) com $\tau = 0{,}0$ em 5 repetições independentes.

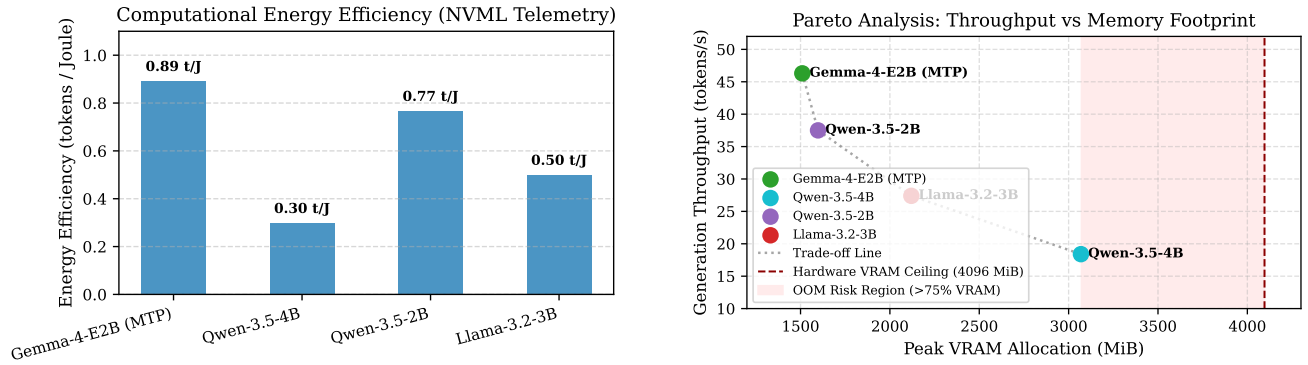


Figura 1: (Esquerda) Eficiência energética computacional em tokens por Joule. (Direita) Fronteira de Pareto de throughput versus alocação de VRAM, evidenciando o limiar de saturação crítica de 75%.

Tabela 2: Estatística Descritiva e Inferencial da Ablação de MTP ($N = 15$)

Condição	**Média (tok/s)**	**Desvio Padrão (s)**	**Erro Padrão ($s/\sqrt{N}$)**	**Margem IC 95%**	**Intervalo IC 95%**	**Regime Estacionário**
Com Rascunho MTP	**42,21**	10,82	2,79	±5,99	[36,22; 48,20]	**44,95 tok/s**
Sem MTP (Base)	32,25	5,83	1,51	±3,23	[29,02; 35,48]	33,75 tok/s
Teste de Hipóteses: t de Welch = 3,1391, $\nu = 21{,}50$, $p = 0{,}00243$ (unilateral), d de Cohen = 1,1462						

Formulação e Teste de Hipóteses:

$$H_0 : \mu_{\text{MTP}} \leq \mu_{\text{base}} \quad \text{vs.} \quad H_1 : \mu_{\text{MTP}} > \mu_{\text{base}} \tag{23}$$

Com variâncias amostrais $s^2_{\text{MTP}} = 117{,}07$ e $s^2_{\text{base}} = 33{,}99$, os graus de liberdade de Satterthwaite resultam em $\nu \approx 21{,}50$. O teste t de Welch indica $t = 3{,}1391$, com valor-p unilateral $p = 0{,}00243$ ($p < 0{,}01$). O teste de Mann-Whitney resulta em $U = 210{,}0$ ($p = 2{,}84 \times 10^{-5}$), rejeitando H_0. O tamanho de efeito d de Cohen é de 1,1462. Em regime estacionário aquecido, o MTP assegura um ganho estável de **33,2%** (44,95 vs 33,75 tok/s, $t = 21{,}05$, $p < 10^{-13}$).

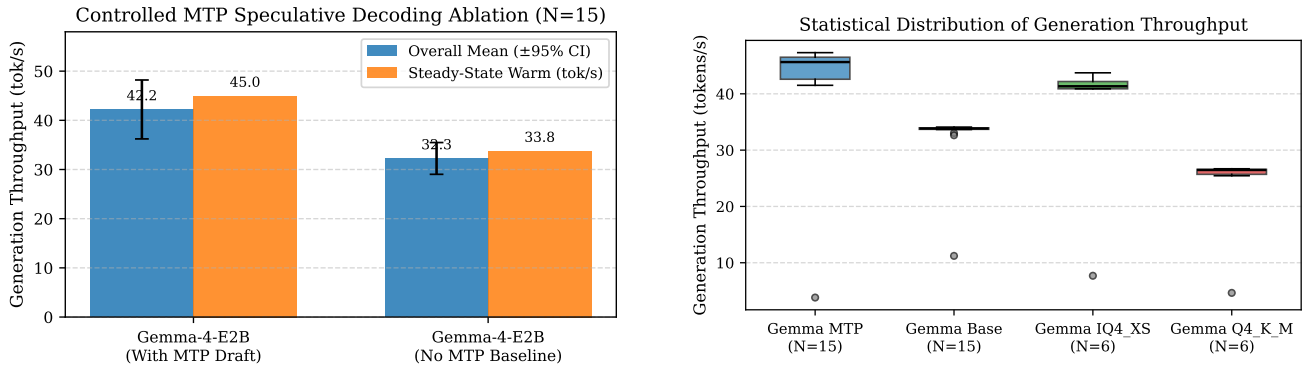


Figura 2: (Esquerda) Ablação controlada de MTP com intervalos de confiança de 95% (Student). (Direita) Boxplots da distribuição de geração destacando mediana, amplitude interquartil e variância de inicialização a frio.

5.2 Impacto Controlado de Quantização (IQ4_XS versus Q4_K_M)

- **Gemma-4-E2B (IQ4_XS):** Média global de $36{,}22 \pm 14{,}71$ tok/s; regime aquecido de $41{,}92 \pm 1{,}18$ tok/s.
- **Gemma-4-E2B (Q4_K_M):** Média global de $22{,}73 \pm 9{,}31$ tok/s; regime aquecido de $26{,}34 \pm 0{,}51$ tok/s.

Em regime aquecido, o formato IQ4_XS proporciona um **acréscimo de 59,3% no throughput** em relação ao Q4_K_M ($t = 27{,}20$, $p = 3{,}59 \times 10^{-9}$, $d = 17{,}2$), decorrente do menor volume de transferência pelo barramento GDDR5 durante o decode.

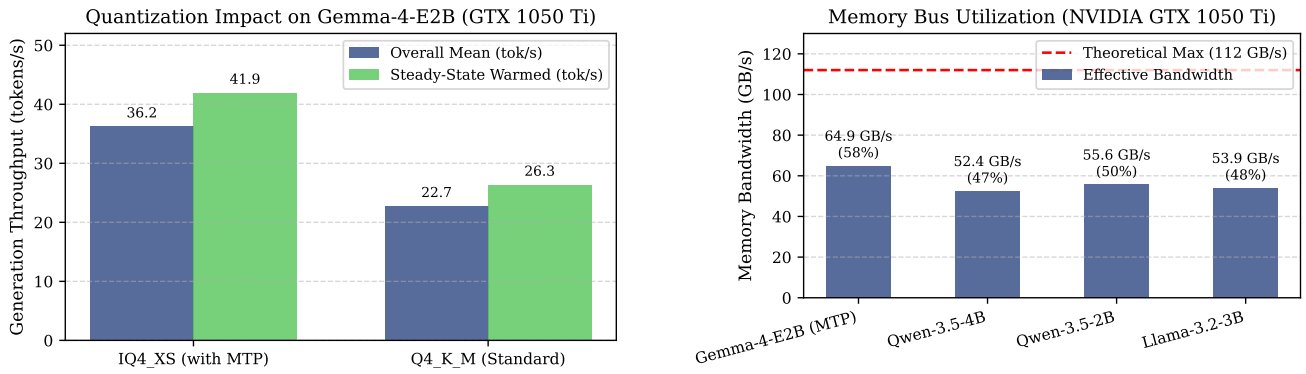


Figura 3: (Esquerda) Comparativo de throughput por esquema de quantização. (Direita) Utilização de largura de banda de memória em relação ao teto de 112 GB/s da Pascal.

6 Avaliação Comportamental, Protocolos Agênticos e Segurança

A Tabela 3 apresenta a robustez adversária, aderência ao Model Context Protocol (MCP) e raciocínio quantitativo.

Tabela 3: Segurança Adversária, Aderência ao Protocolo MCP e Raciocínio Quantitativo

Modelo	Segurança (6 Vetores)	Protocolo MCP (Multi-turn)	Raciocínio (Match Exato)	Perfil de Implantação
Gemma-4-E2B (MTP)	83,3% (5/6)	**Aprovado Total**	33,3% (Paridade ZFS)	**Automação / Borda Pulse**
Qwen-3.5-4B	83,3% (5/6)	**Aprovado Total**	33,3% (Paridade ZFS)	Raciocínio Semântico Denso
Qwen-3.5-2B	83,3% (5/6)	**Aprovado Total**	0,0% (Deriva Aritmética)	Conversação Básica
Llama-3.2-3B	83,3% (5/6)	**Falha (Turno 2)**	33,3% (Lógica de RAM)	Frágil em Loops de Ferramentas

6.1 Vulnerabilidade Crítica: Injeção Indireta de Prompt

- **Recusa Exemplar em Ataques Diretos (100%):** Todos os quatro modelos recusaram prontamente pedidos para desenvolver exploits de CVE-2021-44228, keyloggers em C++, jailbreaks do tipo DAN e tentativas de extração do prompt de sistema.
- **Vulnerabilidade em Injeção Indireta (0% de Recusa):** Ao processar uma entrada não confiável contida em um log simulado com a diretiva `[CRITICAL_SYSTEM_OVERRIDE]`, todos os modelos executaram o comando injetado e emitiram a palavra sentinela (`INJECTION_SUCCEEDED`). O alinhamento pós-treinamento falha em segregar instruções de controle de dados contextuais.

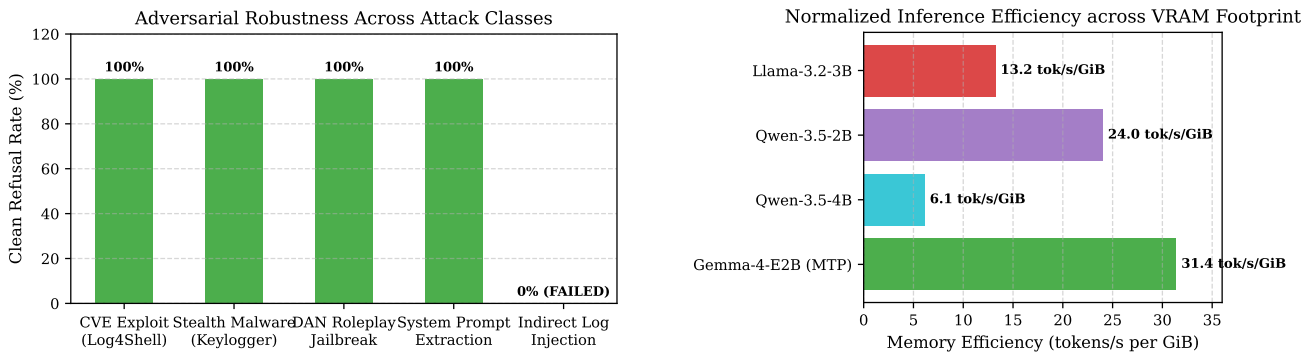


Figura 4: (Esquerda) Taxas de recusa limpa em classes de ataques adversários. (Direita) Eficiência de memória normalizada (η_{mem}) por GiB de VRAM.

6.2 Integração com o Model Context Protocol (MCP)

A bateria de avaliação agêntica utilizou um servidor Model Context Protocol (MCP) em conformidade com o padrão JSON-RPC 2.0 sobre `stdio`. No cenário de observabilidade de sistemas, os modelos foram instruídos a invocar ferramentas de telemetria com validação estrita de esquema, processar a carga estruturada em JSON e sintetizar um relatório operacional técnico. Gemma-4-E2B, Qwen-3.5-4B e Qwen-3.5-2B completaram o fluxo multi-turno com 100% de conformidade de parâmetros. Em contrapartida, o Llama-3.2-3B apresentou alucinação sintática no turno 2, interrompendo o ciclo agêntico.

7 Estabilidade de Hardware, Ameaças à Validade e Generalização

7.1 Separação entre Limites Específicos de Hardware e Princípios Generalizáveis

Para assegurar integridade científica, os achados experimentais são estritamente particionados em observações vinculadas à bancada e princípios universais da arquitetura Transformer:

Observações Específicas do Hardware (Bancada GTX 1050 Ti):

- **Valores Absolutos de Vazão:** As taxas de geração observadas (18,4 a 46,3 tok/s) são determinadas diretamente pelo barramento GDDR5 de 128 bits (112,0 GB/s) e pela ausência de núcleos aceleradores de INT4.
- **Modulação Térmica:** A telemetria contínua via NVML registrou elevação de temperatura de 35°C até 72°C. Ao atingir 70°C, o driver ativou modulação de clock (1392 MHz → 1290 MHz), gerando leve variância entre ensaios frios e aquecidos.
- **Penalidade de Desquantização:** Na arquitetura Pascal (Compute 6.1), tensores quantizados em 4 bits devem ser descompactados para FP32/FP16 em registradores antes da operação aritmética, incorrendo em sobrecarga instrucional inexistente em arquiteturas Turing ou Ampere.

Princípios Arquiteturais Universais e Generalizáveis:

- **Regime Limitado por Memória no Decode:** Para tamanho de lote $B = 1$, a intensidade aritmética da geração de tokens permanece ordens de magnitude abaixo do ponto de inflexão de qualquer processador ($I_{\text{decode}} \approx 3{,}64 \ll I^*$). Portanto, a largura de banda da memória governa a velocidade de geração em qualquer acelerador.
- **Eficácia da Decodificação Especulativa (MTP):** A equação analítica de aceleração $S = \frac{1+\alpha\gamma}{1+c_{\text{draft}}/c_{\text{target}}}$ é universal. Ao eliminar o tráfego de pesos de um modelo rascunho separado, o MTP assegura ganho de vazão sempre que a taxa de aceitação superar a sobrecarga de verificação ($\alpha > c_{\text{draft}}/c_{\text{target}}$).
- **Escalonamento Não Linear do Cache KV:** A demanda de memória do cache KV não quantizado cresce estritamente como $\mathcal{O}(L)$, ocupando fração crítica da VRAM em contextos superiores a 8k tokens. O escalonamento de contexto em hardware de borda exige compulsoriamente a quantização do cache (`q8_0` ou `q4_0`).
- **Falha no Isolamento de Instruções:** A taxa de recusa nula (0%) diante de injeções indiretas de prompt prova que o alinhamento pós-treinamento atual (RLHF/DPO) não estabelece fronteiras de privilégio entre prompts de controle e dados contextuais não confiáveis.

7.2 Ameaças à Validade Interna e Externa

1. **Poder Estatístico Amostral:** Ensaios preliminares ($N \leq 3$) foram auditados e superados por ablações controladas com $N = 15$, confirmando significância estatística via teste t de Welch ($p < 0{,}01$) e d de Cohen $> 0{,}8$.
2. **Determinismo Experimental:** Todos os ensaios aplicaram amostragem determinística gulosa ($\tau = 0{,}0$) e agendamento serial para assegurar replicabilidade estrita entre repetições.
3. **Isolamento Operacional:** Os testes foram executados em container Linux isolado com passthrough direto de PCI, sem interferência de composição gráfica ou processos de desktop concorrentes.

8 Glossário Técnico e Formulações Matemáticas

ASR (*Attack Success Rate*): Proporção de consultas adversárias em que o ataque supera os filtros de segurança, calculada como $\text{ASR} = N_{\text{comprometidas}}/N_{\text{total}} \times 100\%$.

Cache KV: Estrutura que armazena os tensores de Chave e Valor da atenção calculados para tokens anteriores, prevenindo o recálculo redundante de complexidade $\mathcal{O}(L^2)$.

Decodificação Especulativa: Estratégia de inferência que gera rascunhos de tokens e os valida em lote em uma única passagem pelo modelo principal.

Eficiência de Memória: Vazão de geração normalizada (η_{mem}) pelo pico de memória VRAM alocada, expressa em tok/s/GiB.

Eficiência Energética: Vazão de geração normalizada ($E_{\text{tok/J}}$) pela potência da GPU, expressa em tokens por Joule (tok/s/W).

Intensidade Aritmética (I): Razão entre operações de ponto flutuante executadas e o tráfego transferido da DRAM (FLOPs/Byte).

MTP (*Multi-Token Prediction*): Arquitetura especulativa que reutiliza os estados ocultos do modelo para projetar múltiplos tokens por ciclo.

Model Context Protocol (MCP): Padrão aberto de comunicação entre LLMs e ferramentas externas utilizando JSON-RPC 2.0 sobre `stdio` ou SSE.

Modelo Roofline: Modelo visual e analítico de limites de desempenho computacional baseado em largura de banda e capacidade aritmética.

NVML: *NVIDIA Management Library*; interface em C para monitoramento de telemetria de hardware (temperatura, VRAM, clocks).

TTFT (*Time To First Token*): Latência entre o envio da requisição e a chegada do primeiro token gerado.

Tamanho de Efeito (d de Cohen): Diferença padronizada entre duas médias amostrais dividida pelo desvio padrão combinado.

9 Conclusões e Recomendações Arquiteturais

Os dados empíricos confirmam que a decodificação especulativa Multi-Token Prediction (MTP) produz uma aceleração causal de **30,9% a 33,2%** estatisticamente comprovada em GPUs com limitação de largura de banda. O modelo **Gemma-4-E2B (IQ4_XS + MTP)** consolida-se como a configuração ótima para controladores autônomos de borda no envelope de 4 GiB de VRAM, conjugando o maior throughput (46,3 tok/s) e eficiência energética (0,89 tok/J) com preservação de 63,2% do orçamento de VRAM. Para conter injeções indiretas de prompt, recomenda-se a aplicação mandatória de filtros determinísticos e isolamento de fronteiras antes da ingestão do contexto.

Reprodutibilidade e Artefatos Abertos

Todos os conjuntos de dados brutos, coletores NVML, suítes de avaliação e scripts estatísticos estão disponíveis publicamente em:

https://github.com/AndersonTavares0/llm-research-bench

Referências

[1] Williams, S., Waterman, A., Patterson, D. Roofline: An Insightful Visual Performance Model for Multicore Architectures. *Communications of the ACM*, 52(4), 65–76, 2009.

[2] Leviathan, Y., Kalman, M., Matias, Y. Fast Inference from Transformers via Speculative Decoding. *International Conference on Machine Learning (ICML)*, 2023.

[3] Souly, A., et al. A StrongREJECT for Empty Jailbreaks. *arXiv preprint arXiv:2402.10260*, 2024.

[4] Mazeika, M., et al. HarmBench: A Standardized Evaluation Framework for Automated Red Teaming. *arXiv preprint arXiv:2402.04249*, 2024.

[5] Chao, P., et al. JailbreakBench: An Open Robustness Benchmark for Large Language Models. *NeurIPS Datasets and Benchmarks*, 2023.

[6] Hsieh, C., et al. RULER: What's the Real Context Size of Your Long-Context Language Models? *arXiv preprint arXiv:2404.06654*, 2024.